

Interactive User Environment Application on Kubernetes Platform

Chalvatzis Kyriakos



TUC

July 8, 2025



Outline

- 1 Introduction
- 2 Objectives
- 3 Methodology
- 4 Results
- 5 Conclusion



Background

- Data analytics has become essential across all fields — but entry-level users often struggle with the required infrastructure.
- Motivation: Lower the barrier for non-experts to explore and process data using standard tools. User-friendly analytical computing!
- While systems like Apache Spark, Airflow, and Hadoop are powerful, they are built for experienced engineers.
- No current open system provides isolated, user-scoped, web-accessible environments on Kubernetes — tailored for batch analytics.



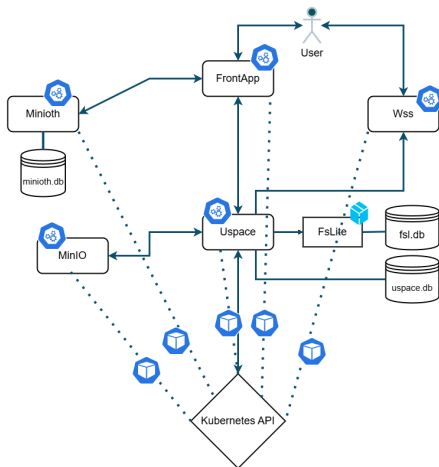
- Although not a research thesis, this work aims to prototype a full-stack system that provides:
 - Secure, multi-user access
 - Data upload and sharing capabilities
 - Batch job execution in a Kubernetes-native way
 - Integrated application-like tools such as DuckDB, Python Pandas, Octave and more can be applied on datasets
- Goal: Deliver an introductory platform for data analytics without overwhelming users.
- Goal: Achieve a Cloud-native and modular design.



- Microservice-based architecture deployed on Kubernetes
- Core components:
 - **Uspace API**: User-facing logic for datasets, jobs, resources
 - **FsLite**: Virtual filesystem abstraction with permissions
 - **MinIO**: Object storage backend
 - **Minioth**: Authentication and group-based access control
 - **FrontApp**: Web frontend (HTML templates, HTMX, WebSocket integration)
- Technologies: Go, Gin, Kubernetes, Docker



System Diagram



Uspace Storage Control

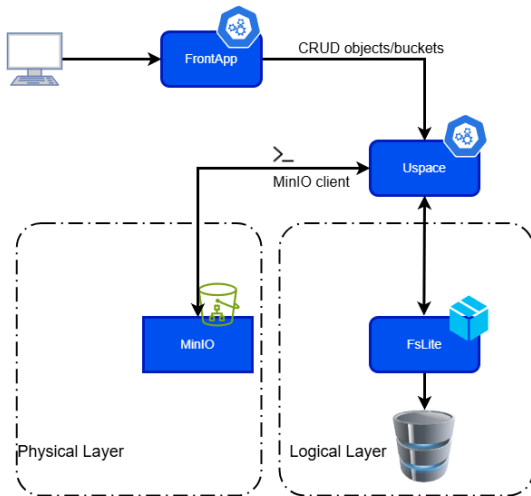


Figure: UspaceStorage



Uspace Job Lifecycle

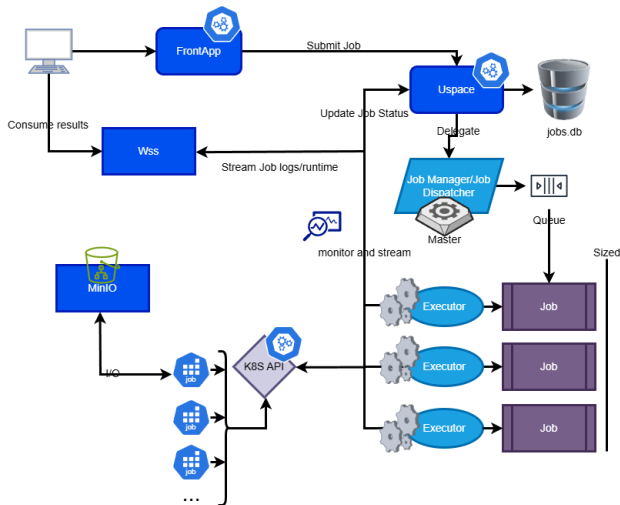


Figure: Uspace Jobs



Demonstration & Key Features

- Fully functioning web interface for:
 - Uploading datasets
 - Launching batch jobs with SQL, Python, or predefined tools
 - Viewing logs in real time (WebSocket streaming)
- Example: Executing DuckDB query on uploaded CSV → result saved and downloadable
- Job isolation: Every job runs in its own Kubernetes Pod



- Jobs run independently across nodes — parallel execution
- Fault isolation through Kubernetes Job controller
- MinIO ensures reliable and persistent dataset storage
- Performance scales with cluster size (pods scheduled dynamically)



- **Accomplishments:**

- Designed and implemented a user-centric data platform on Kubernetes
- Combined identity, authorization, storage, and batch compute layers
- Delivered a fully working demo with real-world tools and users

- **Limitations:**

- **Future Work:**

- Extend toolset: Jupyter, R, Julia, Tool For Tools
- Add CI/CD for job templates and data pipelines
- CLI tool, a road to the Shell
- Distributed orchestration...



Thank you!

kchalvatzis@tuc.gr

